

OpenFocus

A multi-focus registration & fusion workstation — how it works, end to end.

Version 1.30.11 **Stack** Python 3.10+ · PyQt6 · OpenCV · NumPy **Compute** CPU by default · optional CUDA / Apple MPS

Languages English · 简体中文

8

Fusion algorithms, from classical signal processing to a pretrained neural model

3

Registration stages that compose into a single resample of each frame

16-bit

End to end — load, align, fuse and save without narrowing depth

9

Output formats, all lossless by default, with EXIF and XMP carried across

THE RENDER PIPELINE

Every stage below is optional except loading and export. A run can be **fusion only**, **registration only**, or both; the same pipeline serves the interactive window and the batch worker, so a batch job reproduces exactly what the main window is set to.

1 · IMPORT source

A folder of numbered frames, individual files, a video, or a drag-and-drop onto the window, the executable or the dock. Images **JPG PNG BMP TIFF WebP; JXL** with `imagecodecs`; RAW **NEF NRW DNG** with `rawpy`; video **MP4 AVI MOV MKV WMV FLV WebM**, decoded into frames.

2 · LOAD decode & normalise

Frames decode in parallel across cores. JPEG can decode on the GPU via nvJPEG; RAW is developed by LibRaw; JPEG XL gets the whole machine, split between decode workers and libjxl's own pool. The stack is **sized before it is read**, so the thread count is bounded by free memory. Optional downsample on the way in — by percentage or by target long edge.

3 · REGISTER optional

Scale → **Homography** → **ECC**, independent stages that run in pipeline order. Each measures its transform, all of them compose, and the frames are **resampled once** onto a single shared canvas — one Lanczos4 warp, one crop. Anchored to a selectable reference frame.

4 · REGION optional

Full frame, or a selected region of interest. **Crop** saves the region alone; **Paste** fuses the region and blends it back onto a chosen base frame.

5 · FUSE the core

One of eight algorithms decides, per pixel or per band, which frames carry the detail. Frames wider than the tile threshold are processed in overlapping tiles so memory stays bounded; five of the eight methods have a GPU path with automatic CPU fallback.

6 · REFINE optional

IFCNN passes the fused result and the original stack through a convolutional fusion network to recover detail the blending stage missed. Needs PyTorch and the IFCNN checkpoint on disk.

7 · ENHANCE optional · non-destructive

Auto (colour-safe global tone curve) or **Local** (CLAHE), with a strength slider. Applied when the result is previewed and when it is saved — never to the stored render, so it can be changed or removed without fusing again.

8 · EXPORT result

A single image, the aligned stack as a folder or GIF, or a drag straight out of the app. The source frame's **EXIF** is carried across and an **XMP** packet records the version, render date, duration and every option the render used — spliced in without recompressing the pixels.

SUPPORTED SOURCES

FAMILY	EXTENSIONS	REQUIRES	NOTES
Standard images	<code>.jpg</code> <code>.jpeg</code> <code>.png</code> <code>.bmp</code> <code>.tiff</code> <code>.tif</code> <code>.webp</code>	—	16-bit TIFF and PNG enter the pipeline at full depth
JPEG XL	<code>.jxl</code>	<code>imagecodecs</code>	16-bit in, EXIF read from the container's Exif box. Without the package, simply not a supported input
RAW	<code>.nef</code> <code>.nrw</code> <code>.dng</code>	<code>rawpy</code> (LibRaw)	Camera and Adobe DNG both develop. A DNG that OpenFocus wrote is read verbatim, not developed twice
Video	<code>.mp4</code> <code>.avi</code> <code>.mov</code> <code>.mkv</code> <code>.wmv</code> <code>.flv</code> <code>.webm</code>	—	Decoded into an image sequence; every frame becomes a stack entry
Containers	Folders · multi-folder selection	—	Sorted numerically by filename; multiple folders route to batch processing

Entry points: **File** → **Open Folder** (`Ctrl+O`) · **Open Video** (`Ctrl+Shift+O`) · drag onto the window · drag onto the executable, Dock or `.desktop` launcher · command-line arguments.

THE LOADING ENGINE

- **Parallel decode.** Frames load across CPU cores rather than one at a time — roughly 4× on RAW and large stacks.
- **GPU decode.** JPEG can be decoded on the GPU through `nvJPEG`, and RAW developed there too. Toggled under **Settings** → **GPU Image Loading**. JPEG XL has no GPU path and cannot have one — no GPU JXL decoder exists.
- **Sized before read.** RAW and JXL stacks are measured from their headers first, so the app can state the memory the stack will occupy, warn when it is too much, and bound the worker count accordingly.
- **Interruptible.** A load reports progress and stops at its next checkpoint when asked.

BIT DEPTH

Set under **Settings** → **Bit Depth**. The chosen depth holds from load through alignment and fusion to export.

- Auto

Follows the source — engages 16-bit whenever any frame carries more than 8 bits

DEFAULT
- 8-bit

Forces the narrow path; smallest memory footprint
- 16-bit

Forces full depth even from 8-bit sources, leaving headroom for blending

A 16-bit run means no banding in smooth gradients and no quantisation loss where frames are averaged together.

VIEWING & NAVIGATION

Split view

Source frames above, result below, with a draggable splitter. Navigating the source panel moves the result panel to the matching frame where one exists.

Inspection

Mouse-wheel zoom, `Ctrl`+wheel for fine steps, double-click to toggle fit / 100 %, click-drag to pan, and a press-and-hold magnifier for pixel-level checks.

Frame selection

Tick frames in the source list to render a subset — every n -th frame, or an arbitrary selection. The saved metadata records “ k of n frames” when a subset was used.

DOWNSAMPLE ON IMPORT

A dialog appears on every folder open, offering two ways to shrink the stack:

- Scale factor

A percentage applied to every frame
- Long edge

A target in pixels for the longer edge, applied per image — so a mixed-size stack ends on one common long edge

Frames already at or below the target are left alone. Downsampling never enlarges.

STACK TRANSFORMATIONS

Applied to every frame at once, from the **Edit** menu, so alignment between frames is preserved.

Rotate 90° CW

Rotate 90° CCW

Rotate 180°

Flip horizontal

Flip vertical

Resize stack

Labels & annotation

Text labels can be overlaid on the input stack and on the registered stack independently, and removed from either. Overlay colours are authored in 8-bit units and scaled to the working depth, so a label looks the same at either depth.

THE THREE STAGES, IN PIPELINE ORDER



STAGE	WHAT IT CORRECTS	HOW IT FITS	WHEN TO REACH FOR IT
Scale OFF BY DEFAULT	The magnification change a lens introduces as the focus plane moves through the stack — <i>focus breathing</i>	A constrained similarity — uniform scale, rotation, translation — via <code>estimateAffinePartial2D</code> with RANSAC on keypoint matches	When the subject visibly grows or shrinks between the first and last shot. Cancels size drift without the overfitting a full homography risks on blurred frames
Homography	Global perspective change and larger geometric shifts	SIFT/ORB keypoints and RANSAC. Fits <i>both</i> a perspective transform and a constrained similarity per pair, and keeps the perspective one only where it predicts matches it was not fitted to	Handheld captures, or scenes with real perspective change. The model gate means a stack shot on a rail is not bent to fit feature-matching noise
ECC	Residual sub-pixel misalignment	Intensity-based optimisation maximising the enhanced correlation coefficient — no features required	Subtle drift, and as a finishing pass after Homography. Slower but more precise; low-texture frames it can still handle where feature matching cannot

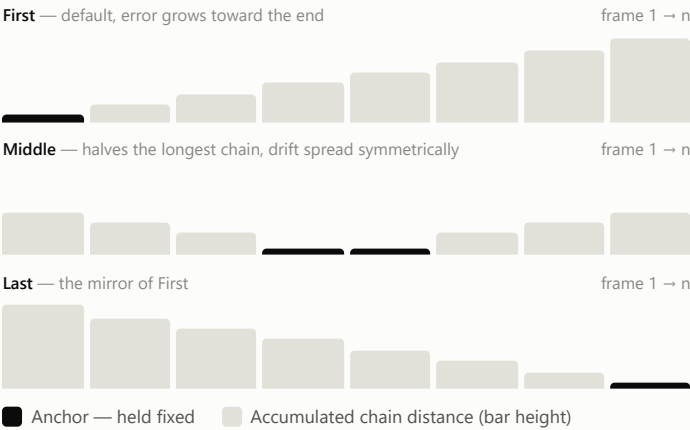
Stages are independent checkboxes. Any combination is valid, and when several are on they always run **Scale → Homography → ECC** regardless of the order you tick them.

Why one warp matters

Each stage measures its own transform on downsampled frames, but the transforms are **composed into a single matrix per frame** before any pixel moves. The stack is then resampled once, with Lanczos4, onto one shared canvas and cropped once to the region every frame covers. A three-stage pipeline that resampled per stage would interpolate three times — softening the frame — and crop three times, throwing away roughly a further tenth of it.

REFERENCE FRAME

Transforms are measured between neighbours and chained back to one fixed frame. Alignment error accumulates with distance from that anchor, so which frame anchors the chain decides where the residual drift ends up. The reference is only cropped, never warped.



Choose **Middle** for long stacks where the far end drifts or ghosts; **Last** when the most important detail sits at the end of the stack.

REGISTRATION SETTINGS

SETTING	DEFAULT	WHAT IT DOES
Downscale width	1024 px	The width frames are resampled to <i>for measuring</i> the transform — never for the output. Lower is faster; setting it to the full frame width aligns at full resolution
Reference frame	First	First · Middle · Last, as shown at left
ECC parallel	On	Computes the ECC pair matrices concurrently. Results are identical either way
Thread count	4	Workers for detection and warping, shared with the rest of the pipeline

When registration is worth running

- Frames were captured handheld, or the rig moved
- The subject changes size across the stack — **enable Scale**
- The scene has parallax, so nearer objects shift between frames
- Results show **ghosting or doubling** — almost always a registration problem, not a fusion one

If registration fails

Try one stage at a time, lower the downscale width, and check the frames have enough contrast and texture for features to be found. Single-channel (greyscale) stacks are handled by all three stages.

4 Fusion algorithms

Eight ways to decide which frame owns which detail. All eight take the same stack and produce one all-in-focus render.

Guided Filter GPU

Spatial domain · edge-preserving weight maps · Li, Kang & Hu 2013

Builds a per-frame weight map from local saliency and refines it with a guided filter, so weights follow real edges instead of blurring across them. Fast, balanced, and forgiving — the historical default.

Kernel 3–51 px, odd · default **31**
Speed  **Fast**
Quality  **Good**

DCT GPU

Frequency domain · block-wise · Haghighat et al. 2011

Measures each block's detail energy in the DCT domain and picks the frame that holds it. The focal-plane map is median-filtered to remove isolated blocks, and neighbouring frames are blended across the block lattice so the grid does not show.

Kernel 3–151 px · default **31** (filters the block map, not pixels)
Block 4 · 8 · 16 · 32 px · default **8**
Plateau Crisp · Balanced · Smooth
Blend On / off across the lattice
Speed  **Medium**
Quality  **Good**

DTCWT GPU

Dual-tree complex wavelets · multi-scale, multi-directional · Lewis et al. 2007

Decomposes each frame into shift-invariant complex wavelet subbands at several scales and six orientations, fuses the coefficients by activity, and reconstructs. Strong on scenes with detail at many different scales.

Kernel Not used — the decomposition sets the scale
Needs `dtcwt` ; GPU path via `pytorch_wavelets`
Speed  **Medium**
Quality  **Very good**

GFG-FG GPU

Gradient focus measure + fast guided filter · Fu et al. 2024

A generalized four-neighbourhood Gaussian gradient operator scores focus, and a fast guided filter refines the resulting weight maps. Cheap to compute and decisive where focus regions are clearly separated.

Kernel 3–51 px, odd · default **7**
Speed  **Fast**
Quality  **Good**

Pyramid RECOMMENDED DEFAULT GPU

Laplacian pyramid, weighted rather than choose-max · after Burt & Adelson 1983

Each frame is split into band-pass detail levels plus a smooth base. Every detail band becomes a weighted mean across the stack, each frame weighted by how far its pooled local energy falls behind the best on offer — a frame twice behind the winner contributes well under a percent, so a genuine focus decision is still a decision, while frames that tie (a background no frame resolves) are averaged instead of picked between. Energies are compared in units of each frame's own grain, so a bright noisy frame cannot win the areas holding no detail. The base band follows the frames that won the detail. Collapsing the pyramid reconstructs the image, and every pixel is finally held inside the range its own frames span, so bands taken from different frames cannot add up to a value no frame had.

Six controls, all exposed and all defaulted to their measured best — see page 5. The textbook choose-max rule remains available as *Selectivity* → *Winner takes all*.

Kernel Energy pooling window · default **5** px (small by design — the pyramid pools again at every level) Speed  **Fast**
Quality  **Very good**

Depth Map — Max CPU ONLY

Per-pixel select from a local Laplacian focus measure

Each pixel is taken whole from the frame whose focus measure is highest there — the classic hard depth map, and an order-independent argmax. Colour and noise never blend across a slice boundary. The family behind Zerene DMap and Helicon methods A/B.

Kernel Focus-measure window · default **9** px
Halo Suppression radius in px · **0 = off**
Speed  **Fast**
Quality  **Very good**

Depth Map — Average CPU ONLY

Contrast-weighted blend from the same focus measure

Frames are blended in proportion to their focus measure, with a small baseline weight. A flat region — where every frame is equally defocused — collapses to the plain mean and recovers the stack's multi-frame SNR, a free $\sqrt{N}$ noise reduction, while textured regions still follow the frame that holds the detail.

Kernel Focus-measure window · default **9** px
Halo Suppression radius in px · **0 = off**
Speed  **Fast**
Quality  **Good**

StackMFF-V4 NEEDS PYTORCH

Pretrained deep model · the AI path

A neural network trained on large multi-focus datasets learns the fusion rule rather than being given one. Highest quality of the eight when the hardware is there. It runs on CUDA or Apple MPS and falls back to CPU automatically; on a card that runs out of memory, the inference batch halves itself and retries, and the size it actually ran at is written into the result's metadata. Inputs are resized to a multiple of 32 for the network's pooling depth, and a preview too small to pool is refused with an explanation rather than producing something wrong.

Batch 1–16 tiles per inference · default **2**, halves itself under VRAM pressure Speed  **Slow**
Kernel Not used — the network decides Quality  **Excellent**

5 Refinement, compute & tuning

Every control here is a trade rather than a right answer — which is why it is exposed. The defaults are what measured best on the project's test stacks.

+ IFCNN refinement — a stage, not a method

A separate checkbox that runs *after* whichever method you chose. The fused result and the original stack go through a fully convolutional fusion network that recovers detail the blending stage missed. It refines a fused image, so it cannot run on its own; it needs PyTorch and the IFCNN checkpoint on disk, and the checkbox greys out with the expected path in its tooltip when either is missing. Results are labelled `<method> + IFCNN`.

COMPUTE PATHS

GPU work goes through PyTorch — CUDA on NVIDIA, MPS on Apple Silicon. Every GPU path falls back to CPU automatically if the device, the library or the memory is not there, and says so in the status console. The status bar shows `CUDA`, `MPS`, `N/A` or `Err`; [Help → Environment Info](#) reports what was actually detected.

GPU: Guided Filter

GPU: DCT

GPU: DTCWT

GPU: GFG-FGF

GPU: Pyramid

GPU: StackMFF-V4

GPU: IFCNN

CPU only: Depth Map

PYRAMID — THE SIX CONTROLS

CONTROL	OPTIONS	DEFAULT	WHAT IT DOES
Kernel slider	3–51 px, odd	5	The window each band's focus energy is pooled over before frames are compared. Small follows fine detail and can chase grain; large decides region by region and rounds off narrow in-focus structures. Small by design — the pyramid pools again at every level, so this window covers twice as much picture at each one
Pyramid levels	Auto, 2–8	Auto	How many band-pass levels each frame is split into. Auto is five, clamped to what the image size can carry. Fewer decide focus on coarser structure; more separate scales finely and cost time
Selectivity	Average · Soft · Balanced · Strict · Winner takes all	Balanced	How sharply each band favours the sharpest frame. Balanced and Strict let a genuinely sharp frame win outright while frames that tie are averaged. Average and Soft blend more of the stack in — smoother backgrounds, softer detail. Winner takes all is the textbook choose-max rule: it takes the single best coefficient everywhere, including in defocused areas where "best" is decided by grain
Scale coherence	Off · Light · Medium · Strong	Off	Makes the coarse levels guide the fine ones, so a pixel cannot take its fine detail from one frame and its coarse structure from another. Reach for it when detail looks like it is sitting on the wrong background. Off by default because it costs sharpness where near meets far
Base band	Mean · Gentle · Balanced · Strong	Balanced	How hard the smooth low-frequency layer follows the frames that won the detail bands. Mean averages every frame and hazes the result when most of the stack is defocused; Strong follows the sharp frames hardest
Ignore grain when nothing is sharp	On / Off	On	Compares frames in units of their own noise, so a bright, grainy, completely defocused frame cannot win the areas where nothing is in focus and stamp its flat tone over them
Keep pixels within the source range	On / Off	On	Holds every pixel between the darkest and brightest value its own frames actually have there. Rebuilding an image from bands taken out of different frames can otherwise produce values no frame had

If a smooth, defocused area comes out crossed by thin dark strokes

The two switches at the bottom of the list are what remove them, and both are **on** by default — check they have not been turned off. Keep Selectivity at *Balanced* or lower rather than *Winner takes all*. If the strokes survive that, try Scale coherence at *Light* or *Medium*.

DCT — THE THREE CONTROLS

CONTROL	DEFAULT	EFFECT
Block size	8 px	The grid every DCT decision is made over — 4, 8, 16 or 32. Smaller follows fine detail and is noisier; larger is steadier but steps harder where near meets far
Plateau	Balanced	How far below the peak energy a frame still counts as in focus on a block. Crisp (0.85) keeps a never-sharp background closer to its sharpest frame; Smooth (0.70) averages more frames there, coming out flatter. Presets rather than a slider — above 0.85 detail-free regions go back to being decided by grain
Blend	On	Blends neighbouring frames across the block lattice. Turning it off copies each block verbatim from a single frame — true source pixels, at the cost of the lattice showing again

DCT's kernel slider median-filters the *focal-plane map*, which holds one entry per block rather than per pixel — so each step reaches eight times further than it does for the pixel-domain methods. That is why its ceiling is 151 rather than 51.

DEPTH MAP — HALO SUPPRESSION

A defocused foreground edge casts a bright glow over the background in exactly the frames where that background is sharp. Plain per-pixel selection copies the glow straight into the result — the classic focus-stacking halo.

With a radius set, a sharply focused region also claims the surrounding band its glow contaminates, so the ring comes out as natural defocused background instead.

Radius 0 Off — bit-identical to not passing the option at all

Set it to Roughly the visible halo width, in pixels

Cost Larger values round off genuine detail near depth edges — the same trade the Radius dial has in Helicon Focus and Zerene Stacker

Use the smallest value that clears the glow. Both Max and Average accept it; Average halos less to begin with, because its blend weights spread.

6 Kernel, tone & memory

The one slider shared across the methods, the enhancement applied after fusion, and how large stacks are kept inside available memory.

KERNEL SIZE AT A GLANCE

METHOD	DEFAULT	RANGE	WHAT THE SLIDER DRIVES
Guided Filter	31	3–51	Filter window
DCT	31	3–151	Median filter on the block map
GFG-FGF	7	3–51	Gradient / guided-filter window
Pyramid	5	3–51	Band energy pooling window
Depth Map (both)	9	3–51	Focus measure pooling window
DTCWT · StackMFF-V4	—	—	Not used by these methods

Values are always odd. Broadly: smaller keeps local detail and more noise; larger is smoother and better at suppressing noise.

Tuning is recorded, not lost

Any control moved off its default is written into the output filename, so two renders that differ only in tuning cannot overwrite each other — and the full set lands in the file's XMP either way. Batch jobs inherit whatever the main window is set to.

TILING — THE MEMORY STRATEGY

Large frames are fused in overlapping tiles rather than whole, because GPU memory demand scales with tile *area* and with the square of the stack size. The overlap is what keeps the seams invisible.

SETTING	DEFAULT	RANGE	EFFECT
Block size	1024 px	64 – 16384	Smaller = less memory, slower
Overlap	256 px	up to 4096	Raise it if seams show
Threshold	2048 px	256 – 131072	Frame size above which tiling engages at all

How many tiles run at once is derived from free memory at render time, not from the thread count — and forced to one for the GPU methods, which are already parallel inside a tile. A render that tiled records both facts in its metadata.

CONTRAST ENHANCEMENT

A post-fusion stage, off by default. It is applied when the result is displayed and again when it is saved — never to the stored render — so it previews live and can be changed or removed without fusing again.

Auto	A colour-safe global tone curve. Brightness moves through a luminance gain, so hue is untouched
Local	CLAHE — contrast limited adaptive histogram equalisation, working region by region
Strength	0–100 %. At 0, or with the method off, the input is returned unchanged

Depth-aware: a 16-bit result stays real 16-bit data, not 8-bit values widened into a bigger type. The setting in force at save time is what gets written into the result's metadata.

PERFORMANCE SETTINGS

SETTING	DEFAULT	WHAT IT CONTROLS
Thread count	4	Workers for loading, detection, warping and CPU fusion. 1–256; match your core count
GPU image loading	On	nvJPEG decode and GPU RAW develop on import
StackMFF batch	2	Tiles per inference, 1–16; halves itself under VRAM pressure
ECC parallel	On	Concurrent ECC pair matrices; identical results

Out of memory?

Enable or tighten tiling, reduce the block size, use a smaller kernel, or downsample the stack on import. For CUDA specifically, lower the StackMFF batch size — though it already halves itself and retries on an allocator failure.

7 Working modes, batch & output

What a single run can be asked to do, how to queue many of them, and the nine ways a result leaves the application.

THREE WAYS TO RUN A JOB

Registration and fusion are independent selections, so a run is whichever of them you tick:

Fusion only	No registration stage selected. The frames are taken as they are — correct for a stack shot on a rigid rail or a microscope stage
Registration only	No fusion method selected. The output is the <i>aligned stack</i> , savable as a folder or a GIF. Recorded in metadata as "None (registration only)"
Both	The frames are aligned, then fused. The aligned stack is cached and reused if you re-render with the same registration settings, so only the fusion runs again

Changing any registration setting invalidates that cache — the alignment is only reused on an exact match of stages and reference frame.

BATCH PROCESSING

Queue any number of folders and leave it running. Each folder is processed independently, with live progress and cancellation at any point.

Settings	Inherited from the main window — the same fusion method, kernel, tuning, registration stages, contrast, bit depth and tiling
Output	Into a subfolder of each input, alongside the input, or into one custom folder
Format	Chosen per batch from the same list as the save dialogs; the choice is remembered
Naming	Timestamp plus the parameters that were used, so runs never collide

Dropping several folders onto the window asks whether they are one stack or many, and routes to batch when they are many.

OUTPUT FORMATS

FORMAT	DEPTH	COMPRESSION	METADATA	NOTES
PNG	8 / 16-bit	Lossless	EXIF + XMP	16-bit stays 16-bit — the metadata is spliced in without recompressing
JPEG XL	8 / 16-bit	Lossless	EXIF + XMP	Typically half the size of the equivalent PNG. Needs <code>imagecodecs</code> ; not offered without it
TIFF	8 / 16-bit	Lossless	EXIF	Widest compatibility for a high-depth deliverable
DNG	16-bit	None · lossless · lossy	EXIF + camera tags	Linear demosaiced DNG for raw-converter workflows — see page 8
WebP	8-bit	Lossless	—	A 16-bit render is narrowed on the way out. Hard limit of 16383 px per side, reported as itself
JPG	8-bit	Lossy	EXIF + XMP	Good quality-to-size ratio; the default for drag-out
BMP	8-bit	None	—	Uncompressed, maximum compatibility
GIF	8-bit	Lossy palette	—	Animation of an input or registered stack, not a fused result
Folder	as chosen	as chosen	as chosen	A whole stack written out frame by frame under its own filenames

REGION OF INTEREST

Fuse a part of the frame instead of all of it — much faster when you are comparing settings, and the only way to treat one subject differently from its surroundings.

1 · Enable	The ROI button. Needs at least two frames loaded; the stack is aligned with ECC first, automatically, so the region means the same thing in every frame
2 · Draw	Drag a rectangle on the result panel
3 · Render	A dialog asks how to output it
Crop	The result is a small image containing only the selected region
Overlay	The result is a full-size image — a chosen base frame with the fused region pasted onto it

WAYS OUT OF THE APP

- **Save result** (`Ctrl+S`) — one fused image, format from the dialog
- **Save stack** (`Ctrl+Shift+S`) — the registered or the input stack, as a folder of images or as an animated GIF
- **Drag out** — drag a result from the output list straight into another application
- **Batch output** — written automatically, one result per input folder

Interrupting a render

The **Stop** button activates for the duration of a render. It unwinds at the next safe checkpoint — between pipeline stages, or between tiles on large images — the controls unlock and no result is produced. On a heavily tiled image this takes a moment while tiles already in flight finish.

DNG OUTPUT SETTINGS

An uncompressed *linear* (already demosaiced) DNG at full 16-bit, tagged with an already-neutral white balance and the transfer curve the develop actually applies — so Lightroom or RawTherapee renders the result you saw, not a second tone curve on top of it. Reloading one into OpenFocus returns the exact pixels.

Compression	None (default) · Lossless Huffman JPEG — identical pixels at about half the size · Lossy baseline JPEG with a quality setting, 8-bit only, meant for proxies
Fast-load preview	Embeds a half-resolution JPEG in a SubIFD, so browsers and raw converters draw a thumbnail from a few hundred kilobytes instead of decoding the full linear raw
Colour space	sRGB primaries, or the source camera's own colorimetry where the file can supply it

The lossless mode is only offered where `imagecodecs` can read it back — a result that cannot be reopened is not a result. `Make` and `Model` stay OpenFocus's own, because those are what a converter reads to pick a camera profile.

TROUBLESHOOTING — SYMPTOM TO SETTING

SYMPTOM	WHAT TO CHANGE
Result is blurry, ghosted or doubled	A registration problem, not a fusion one. Enable Homography and/or ECC ; if the subject changes size across the stack, add Scale . On a long stack, set the reference frame to Middle
Bright halo around the subject	Switch to a Depth Map method and raise Halo suppression to roughly the halo's width in pixels. Use the smallest value that clears it
Thin dark strokes over a smooth background	A Pyramid symptom. Confirm both bottom switches are on , keep Selectivity at Balanced or lower, then try Scale coherence at Light or Medium
Visible seams or a block lattice	Raise the tile overlap . On DCT specifically, leave Blend on and raise the kernel to filter the block map harder
Out of memory	Enable or tighten tiling, reduce the block size, downsample on import, or lower the StackMFF batch size
Registration fails or drifts	Run one stage at a time, lower the downscale width , and check the frames carry enough contrast and texture for features to be found
StackMFF-V4 greyed out	<code>pip install torch torchvision</code> ; for GPU, a CUDA-enabled build. Confirm with Help → Environment Info
JXL or RAW not offered	<code>pip install imagecodecs</code> for JPEG XL, <code>pip install rawpy</code> for NEF/NRW/DNG. Without them those formats are simply not supported inputs
Processing is slow	Raise the thread count, enable GPU loading, pick a cheaper method (Guided Filter or Pyramid over StackMFF-V4), or downsample on import. Stop interrupts a run at its next checkpoint

KEYBOARD

Files

<code>Ctrl+O</code>	Open folder
<code>Ctrl+Shift+O</code>	Open video
<code>Ctrl+S</code>	Save result
<code>Ctrl+Shift+S</code>	Save registered stack
<code>Ctrl+W</code>	Clear stack
<code>Ctrl+Q</code>	Exit

Navigation

<code>←</code> <code>→</code>	Previous / next frame
Wheel	Zoom
<code>Ctrl</code> + wheel	Fine zoom
Double-click	Fit ↔ 100 %
<code>Space</code>	Pan mode
<code>Delete</code>	Delete selected

Where settings live

All of it persists to `openfocus.cfg.json` beside the application — including the fusion method and its tuning, registration stages, tiling, threads, bit depth, DNG options, contrast, language, recent folders and the last output format. **File** → **Save All Settings** writes it on demand.